

REPASO INTEGRADOR · PARTE 1 DE 3

Programación Orientada a Objetos en C++

Repaso Pre-Parcial

Caso de estudio: Sistema de Biblioteca — Módulos 1 a 3

```
class ObjetoDeEstudio {  
private:  
    Conocimiento conocimiento;  
public:  
    void prepararParcial();  
};
```

Introducción a la POO · Encapsulamiento · Relaciones entre clases

Programación Orientada a Objetos - Leonardo Brambilla - FCyT - UADER

MÓDULO

1

≈ 20 minutos

Introducción a la POO

Objetivos de esta sección

- Explicar qué es la Programación Orientada a Objetos y por qué la usamos.
- Diferenciar correctamente clase y objeto.
- Identificar estado, comportamiento e identidad en un objeto.

¿Qué es la Programación Orientada a Objetos?

- ▶ **Paradigma** que organiza el software en unidades llamadas **objetos**, cada una con datos y comportamiento propio.
- ▶ Modela el problema en términos de "**entidades**" del dominio (Libro) en lugar de una secuencia de instrucciones sueltas.
- ▶ Se apoya en cuatro pilares:
 - encapsulamiento,
 - herencia,
 - polimorfismo
 - y abstracción.
- ▶ Beneficios: mayor organización, reutilización de código, mantenibilidad y modelado más cercano a la realidad.

Paradigma procedural vs. orientado a objetos

PROCEDURAL	POO
-----	-----
datos sueltos	+-----+
funciones sueltas	Libro
	- titulo
f(datos) --->	- disponible
g(datos) --->	+prestar()
	+-----+

El objeto agrupa DATOS y COMPORTAMIENTO juntos.

? En el sistema de Biblioteca, ¿qué entidades del mundo real serían buenas candidatas a convertirse en clases?

Clase vs. Objeto

- ▶ Una **clase** es un tipo de dato definido por el usuario:
 - describe qué **atributos**
 - y qué **operaciones** tendrán sus instancias.
- ▶ Un **objeto** es una instancia concreta de una clase: ocupa memoria real y tiene sus propios valores de atributos.
- ▶ Igual que 'int' es un tipo y '5' es un valor de ese tipo, 'Libro' es un tipo y 'elQuijote' es un objeto de ese tipo.
- ▶ Se pueden crear **muchos objetos** distintos a partir de una misma clase.

libro.h / main.cpp

```
class Libro {           // CLASE
public:
    string titulo;
    bool disponible;
};

int main() {
    Libro elQuijote;      // OBJETO 1
    Libro cienAnios;      // OBJETO 2
    elQuijote.titulo = "El Quijote";
    cienAnios.titulo = "Cien años de soledad";
}
```

 **Error frecuente:** Decir que "la clase es un plano y el objeto es la casa" -> la clase es un **TIPO** y el objeto una **INSTANCIA** de ese tipo.

? Si declaro 'Libro a, b;', ¿cuántas clases y cuántos objetos existen en memoria?

Estado, comportamiento e identidad

- ▶ **Estado:** los valores actuales de los atributos del objeto en un momento dado (título, disponible).
- ▶ **Comportamiento:** las operaciones que el objeto puede realizar, definidas por sus métodos (prestar(), devolver()).
- ▶ **Identidad:** lo que distingue a un objeto de otro aunque tengan el mismo estado (su dirección de memoria / identidad propia).
- ▶ Dos objetos Libro pueden tener el mismo título y aun así ser objetos distintos.

Ejemplo: Socio

```
class Socio {  
private:  
    string nombre;        // estado  
    int librosPrestados; // estado  
public:  
    void pedirPrestamo() { // comportamiento  
        librosPrestados++;  
    }  
};  
  
Socio s1, s2; // identidades distintas  
// s1 y s2 pueden tener igual estado  
// pero NO son el mismo objeto
```

? ¿Pueden dos objetos Socio distintos tener exactamente el mismo estado? ¿Siguen siendo objetos diferentes?

Resumen — Introducción a la POO

✓ Ideas clave

- La POO organiza el software en objetos que combinan datos y comportamiento.
- Clase = tipo definido por el usuario. Objeto = instancia concreta de ese tipo.
- Todo objeto tiene estado, comportamiento e identidad propios.
- La clase define la estructura y el comportamiento, mientras que cada objeto posee su propio estado (sus propios valores para los atributos).

⚠ Errores frecuentes

- Confundir **clase** con **objeto** y usarlos como sinónimos.
- Usar la analogía "clase = plano/molde" sin aclarar la relación tipo–instancia.
- Pensar que dos objetos con igual estado son 'el mismo objeto'.

📝 Ejercicio rápido

Escribí la declaración de una clase Prestamo (sin implementar métodos) con al menos dos atributos coherentes con el Sistema de Biblioteca, y luego declará dos objetos de esa clase.

MÓDULO

2

≈ 30 minutos

Encapsulamiento

Objetivos de esta sección

- Aplicar los modificadores de acceso private, protected y public.
- Implementar getters, setters, constructores y destructor.
- Comprender el uso del puntero this.

Atributos, métodos y modificadores de acceso

- ▶ **Encapsulamiento:** **ocultar** el estado interno del objeto y exponer **sólo** lo necesario mediante métodos.
 - **private:** accesible únicamente dentro de la propia clase.
 - **protected:** accesible en la propia clase y en las clases derivadas (se retoma en Módulo 4).
 - **public:** accesible desde cualquier parte del programa.
 - c++ permite el uso de **friend**, que pueden acceder a miembros private y protected sin formar parte de la clase. Reducen el nivel de encapsulamiento.

Modificadores de acceso

Modificador	Misma clase	Clase derivada	Exterior
private	Sí	No	No
protected	Sí	Sí	No
public	Sí	Sí	Sí

⚠ **Error frecuente:** Declarar todos los atributos como **public** "para que sea más fácil", rompiendo el encapsulamiento.

? ¿Por qué el atributo 'disponible' de Libro debería ser private y no public?

Getters y Setters

- ▶ **Getters:** métodos públicos que permiten leer un atributo privado sin exponerlo directamente.
- ▶ **Setters:** métodos públicos que permiten modificar un atributo privado, validando el nuevo valor.
- ▶ Un setter puede rechazar valores inválidos; un atributo public nunca podría hacerlo.
- ▶ No todo atributo necesita setter: algunos deben ser de solo lectura desde afuera.

libro.h

```
class Libro {  
private:  
    string titulo;  
    int anioPublicacion;  
public:  
    string getTitulo() const {  
        return titulo;  
    }  
    void setAnio(int anio) {  
        if (anio > 0 && anio <= 2026)  
            anioPublicacion = anio;  
    }  
};
```

? ¿Qué validación agregarías a un `setNombre(string)` para la clase `Socio`?

Constructores y destructor

- ▶ **Constructor:** método especial que inicializa el objeto en el momento de su creación; tiene el mismo nombre que la clase.
- ▶ **Constructor por inicialización:** recibe parámetros para fijar el estado inicial.
- ▶ **Destructor** (~NombreClase()): se ejecuta automáticamente cuando el objeto deja de existir; libera recursos.
- ▶ Si no se define ningún constructor, el compilador genera uno por defecto (sin parámetros).
 - Libro() = default;

libro.h

```
class Libro {
private:
    string titulo;
    bool disponible;
public:
    Libro(string t) : titulo(t),
                    disponible(true) {}

    ó
    Libro(string titulo) {
        this->titulo = titulo;
        this->disponible = true;
    }

    ~Libro() {
        // libera recursos si los hubiera
    }
};

Libro l1("El Quijote"); // se construye
// ...
// al salir de alcance -> ~Libro()
```

⚠ **Error frecuente:** Olvidar la lista de inicialización y asignar los valores dentro del cuerpo del constructor, lo cual es menos eficiente y falla con atributos const o referencias.

? ¿Qué diferencia hay entre 'Libro(string t) : titulo(t) {}' y 'Libro(string t) { titulo = t; }'?

El puntero this

- ▶ **this** es un puntero implícito que apunta al propio objeto sobre el cual se invocó el método.
- ▶ Se usa para distinguir un atributo de un parámetro con el mismo nombre.
- ▶ Todo método no estático recibe this de forma automática e implícita.
- ▶ Como this es un puntero, para acceder a un atributo o método debemos utilizar el operador ->.
 - (*this).nombre = nombre;
 - this->nombre = nombre;

socio.h

```
class Socio {  
private:  
    string nombre;  
public:  
    void setNombre(string nombre) {  
        this->nombre = nombre;  
        //   ^ atributo   ^ parámetro  
    }  
    Socio& agregarPrestamo() {  
        // ... lógica  
        return *this;    // encadenamiento  
    }  
};
```

? ¿Qué imprimiría 'cout << this;' dentro de un método? ¿Qué tipo de dato es this?

Ejemplo integrador: clase Libro completa

- ▶ Reúne en un solo ejemplo: atributos privados, constructor por inicialización, getters/setters y destructor.
- ▶ Observar cómo el estado interno (disponible) sólo puede cambiar a través de prestar()/devolver().
- ▶ Este es el diseño que usaremos como base para las relaciones entre clases del Módulo 3.

libro.h — versión final

```
class Libro {  
private:  
    string titulo;  
    string autor;  
    bool disponible;  
public:  
    Libro(string t, string a)  
        : titulo(t), autor(a),  
          disponible(true) {}  
    string getTitulo() const { return titulo; }  
    bool estaDisponible() const { return disponible; }  
    void prestar() { disponible = false; }  
    void devolver() { disponible = true; }  
    ~Libro() {}  
};
```

? ¿Por qué prestar() y devolver() son mejores que exponer 'disponible' como público con un setter genérico?

Resumen — Encapsulamiento

✓ Ideas clave

- `private/protected/public` controlan la visibilidad de atributos y métodos.
- Getters y setters exponen el estado de forma controlada y validada.
- Constructor inicializa el objeto; destructor libera recursos al finalizar su vida útil.
- `this` identifica al objeto actual dentro de sus propios métodos.

⚠ Errores frecuentes

- Atributos declarados `public` por comodidad.
- Asignar en el cuerpo del constructor en vez de usar la lista de inicialización.
- Olvidar el destructor cuando la clase gestiona memoria dinámica (se retoma en Módulo 6-7).
- Confundir `this` (puntero) con `*this` (objeto).

🔑 Ejercicio rápido

Agregá a la clase `Socio` un constructor por inicialización, un getter de nombre y un setter validado para un atributo `'email'` (debe contener `'@'`).

MÓDULO

3

≈ 30 minutos

Relaciones entre clases

Objetivos de esta sección

- Identificar correctamente asociación, agregación, composición y herencia.
- Representar cada relación con UML.
- Distinguir cuándo usar cada tipo de relación en el diseño.

Asociación

- ▶ Definición: vínculo general en el que un objeto usa o conoce a otro, sin control sobre su ciclo de vida.
- ▶ Ejemplo: un Socio conoce a un Bibliotecario que lo atendió, pero ninguno depende del otro para existir.
- ▶ Ventaja: bajo acoplamiento, ambos objetos son independientes.
- ▶ Se usa cuando dos clases colaboran pero cada una tiene vida propia.

UML — Asociación

```
+-----+           +-----+
| Socio | ----- | Bibliotecario |
+-----+           +-----+
          1           1

class Socio {
    Bibliotecario* atendidoPor;
};
// Socio NO crea ni destruye
// al Bibliotecario
```

? ¿La relación entre un Prestamo y el Socio que lo solicitó es asociación? ¿Por qué?

Agregación

- ▶ Definición: relación "todo-parte" débil; la parte puede existir independientemente del todo.
- ▶ Ejemplo: una Biblioteca tiene Socios, pero un Socio puede existir (y afiliarse a otra biblioteca) aunque esta biblioteca cierre.
- ▶ Se representa en UML con un rombo hueco del lado del "todo".
- ▶ Ventaja: reutilización de objetos entre distintos 'todos'.
- ▶ Desventaja: gestión de ciclo de vida menos clara.

UML — Agregación

```
+-----+ o---- +-----+
| Biblioteca |      | Socio |
+-----+      +-----+
```

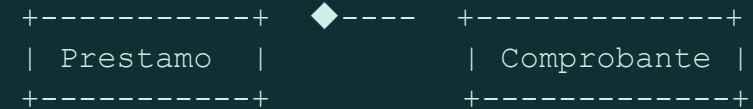
```
class Biblioteca {
    vector<Socio*> socios;
};
// los Socio* pueden vivir
// fuera de la Biblioteca
```

? Si se elimina el objeto Biblioteca, ¿deberían destruirse también los objetos Socio? ¿Por qué eso confirma que es agregación?

Composición

- ▶ Definición: relación "todo-parte" fuerte; la parte NO puede existir sin el todo (ciclo de vida compartido).
- ▶ Ejemplo: un Prestamo tiene un ComprobanteDePago; si se elimina el Prestamo, el comprobante no tiene sentido por sí solo.
- ▶ Se representa en UML con un rombo relleno del lado del "todo".
- ▶ El todo suele crear y destruir a la parte (a menudo como atributo por valor, no por puntero).

UML — Composición



```
class Prestamo {
    Comprobante comprobante; // valor
};
// Comprobante muere junto
// con el Prestamo
```

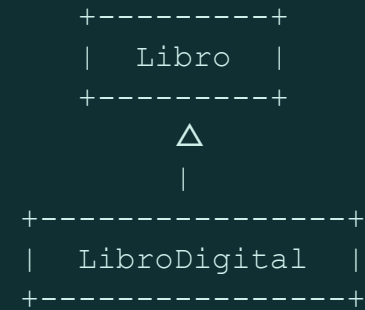
⚠ **Error frecuente:** Confundir composición con herencia: composición es "TIENE UN", herencia es "ES UN".

? ¿Por qué un atributo declarado por valor (no puntero) sugiere composición en vez de agregación?

Herencia (introducción)

- ▶ Definición: mecanismo por el cual una clase (derivada) adquiere atributos y métodos de otra (base), modelando una relación ES-UN.
- ▶ Ejemplo: un LibroDigital ES UN Libro, con características adicionales (tamaño en MB, formato).
- ▶ Debe usarse cuando existe una verdadera relación ES-UN y se necesita polimorfismo, no sólo para reutilizar código.
- ▶ Se profundiza junto con polimorfismo en el Módulo 4.

UML — Herencia



```
class LibroDigital : public Libro {
    double tamanoMB;
};
```

⚠ **Error frecuente:** Usar herencia solamente porque "ahorra escribir código", sin que exista una verdadera relación ES-UN.

? ¿'Bibliotecario ES UN Socio' o más bien ambos deberían derivar de una clase común 'Persona'?

Tabla comparativa de relaciones

- ▶ Las cuatro relaciones permiten reutilizar diseño, pero con distinto grado de acoplamiento y dependencia de ciclo de vida.
- ▶ De más débil a más fuerte:
 - Asociación → Agregación → Composición → Herencia.
- ▶ La pregunta clave para elegir:
 - ¿es ES-UN (herencia) o TIENE-UN (las otras tres)?
 - Y si es TIENE-UN, ¿la parte sobrevive al todo?

Comparación

Relación	UML	Ciclo de vida	Ejemplo Biblioteca
Asociación	línea simple	independiente	Socio — Bibliotecario
Agregación	rombo hueco	parte sobrevive	Biblioteca — Socio
Composición	rombo relleno	parte no sobrevive	Prestamo — Comprobante
Herencia	triángulo	ES-UN, no aplica	Libro — LibroDigital

? Ubicá la relación 'Prestamo — Libro' (un préstamo referencia a un libro que sigue existiendo en el catálogo). ¿Cuál de las cuatro es?

Resumen — Relaciones entre clases

✓ Ideas clave

- Asociación: colaboración sin dependencia de ciclo de vida.
- Agregación: todo-parte débil (rombo hueco); la parte sobrevive al todo.
- Composición: todo-parte fuerte (rombo relleno); la parte no sobrevive al todo.
- Herencia: relación ES-UN que habilita el polimorfismo (Módulo 4).

⚠ Errores frecuentes

- Confundir agregación con composición por no analizar el ciclo de vida.
- Usar herencia para relaciones que en realidad son TIENE-UN.
- Pensar que todas las relaciones son "lo mismo" porque todas reutilizan código.

📝 Ejercicio rápido

Para el Sistema de Biblioteca, clasifica la relación entre 'Estanteria' y 'Libro': ¿asociación, agregación o composición? Justifícalo pensando en el ciclo de vida.